HDOC Day-50

Write a C program that reads the *size* and elements of a variable-length integer array, *nums[]*. Pass the array to a function named *majority_element()* using its address. Use pointers to pass the array between the calling function and *majority_element()*. The function *majority_element()* must:

1. Count the occurrences of each distinct element in *nums[]*.
2. Find the element that appears strictly > ⌊size / 2⌋ times.
3. Display the majority element if it exists.
4. Display -1 if no majority element exists.

Note: The majority element is not necessarily the element with the highest frequency. It must occur strictly > ⌊size / 2⌋ times.

Example 1:

Array: 3 3 2 3 3

Output:

Majority element: 3

Explanation: The element 3 occurs four times. Here, ⌊5 / 2⌋ = 2. As the element 3 occurs strictly >2 times, it is the majority element.

Example 2:

Array: 2 2 1 1 1 2 2

Output:

Majority element: 2

Explanation: The element 2 occurs four times. Here, ⌊7 / 2⌋ = 3. As the element 2 occurs strictly >3 times, it is the majority element.

Example 3:

Array: 1 2 3 2 3 3

Output:

-1

Explanation: The element 3 occurs three times. Here, ⌊6 / 2⌋ = 3. As the element 3 does not occur strictly >3 times, no majority element exists.

1. Algorithm

1. Start.
2. Read the size n of the array.
3. Read n integer elements into nums[].
4. Pass the array address and size to majority_element() using pointers.
5. Inside the function, count the occurrences of each distinct element.
6. Check whether any element occurs more than n/2 times.
7. If found, display the majority element.
8. Otherwise, display -1.
9. Stop.

2. C Program

```
#include <stdio.h>


int majority_element(int *nums, int n)
{
   int i, j, count;


   for (i = 0; i < n; i++)
   {
      count = 0;


      // Skip elements already checked
      for (j = 0; j < i; j++)
      {
         if (*(nums + i) == *(nums + j))
```

```
            break;
        }

        if (j < i)
            continue;

        // Count occurrences of the current element
        for (j = 0; j < n; j++)
        {
            if (*(nums + i) == *(nums + j))
                count++;
        }

        // Check majority condition
        if (count > n / 2)
            return *(nums + i);
    }

    return -1;
}

int main()
{
    int n, i, result;

    printf("Enter array size: ");
    scanf("%d", &n);
```

```
    if (n <= 0)
    {
        printf("Invalid array size\n");
        return 0;
    }

    int nums[n];

    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    result = majority_element(nums, n);

    if (result != -1)
        printf("Majority element: %d\n", result);
    else
        printf("-1\n");

    return 0;
}
```

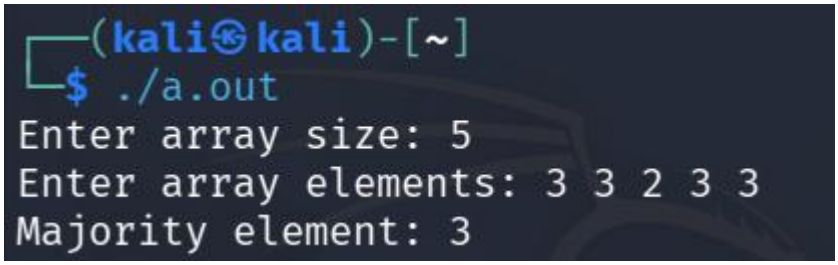
Test Case Table:

Test Case	Input array	Expected Output
1	3 3 2 3 3	Majority element: 3
2	2 2 1 1 1 2 2	Majority element: 2
3	1 2 3 2 3 3	-1
4	5 5 5 5	Majority element: 5
5	1 2 3 4	-1

Compilation and Execution:

The following program has been compiled and executed successfully.

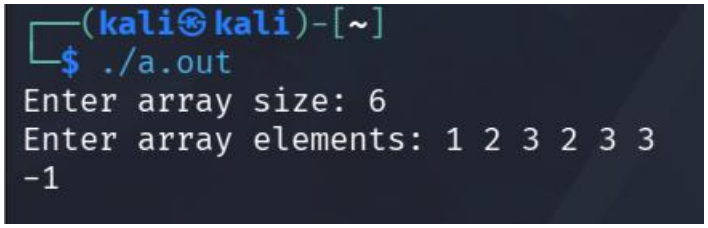
Test Case 1:



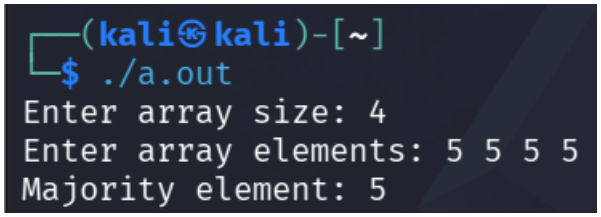
Test Case 2:



Test Case 3:



Test Case 4:



Test Case 5:

